

# The FESOM SSH preconditioner is not symmetric

Response, and a suggested fix for `solver.F90`

**Date** 6 August 2026

**Re** “the MITgcm preconditioner is symmetric by construction — and this ought to destroy plain CG in

**Detail** `docs/plans/20260805-m10-ssh-derivations.md §0.4`, branch `m10-ssh-solvers`

## 1. Short answer

Two questions were raised. Both have answers, and the second one is the interesting one.

1. **Is it the Kokkos port’s bug?** No — the port reproduces `solver.F90` term for term. The asymmetry is in the Fortran.
2. **Should this not destroy plain CG as well?** No, and for a specific reason: plain PCG never uses the identity that fails. It loses optimality, not soundness — measured cost  $\approx 7\%$  **more iterations**, not divergence. The communication-avoiding variants have no such protection and diverge outright.

## 2. Where the asymmetry is

`src/solver.F90`, in `ssh_solve_preconditioner`:

```
pr_values(offset+1) = 1.0_WP/ssh_stiff%values(offset+1)
DO n=2, nend
  node = cind(offset+n)
  pr_values(n+offset) = -0.5_WP*(ssh_stiff%values(n+offset)/ssh_stiff%values(1+offset))/ &
    (ssh_stiff%values(1+offset) + diag_values(node))
END DO
```

That is

$$\text{pr}[i, i] = \frac{1}{d_i}, \quad \text{pr}[i, j] = -\frac{1}{2} \frac{a_{ij}/d_i}{d_i + d_j} = -\frac{1}{2} \frac{a_{ij}}{d_i (d_i + d_j)}, \quad d_i \equiv A_{ii}. \quad (1)$$

The factor  $(d_i + d_j)$  is symmetric and  $a_{ij} = a_{ji}$ , so **the only asymmetric part is the leading  $1/d_i$** : the transpose entry carries  $1/d_j$  instead, giving  $\text{pr}[i, j]/\text{pr}[j, i] = d_j/d_i$ .

Writing  $M^{-1} = D^{-1}C$  with  $D = \text{diag}(A)$ ,  $C_{ii} = 1$  and  $C_{ij} = -\frac{1}{2}a_{ij}/(d_i + d_j)$ : the matrix  $C$  is symmetric. The asymmetry sits entirely in the left factor  $D^{-1}$ .

Measured on CORE2:  $\max |M_{ij} - M_{ji}| / \max |M_{ij}| = \mathbf{0.638}$ . The same measurement on  $A$  gives  $1.4 \times 10^{-13}$ , so the instrument is sound —  $A$  is symmetric,  $M^{-1}$  is not.

## 3. Why plain CG survives

Plain PCG computes

$$\alpha_i = \frac{\gamma_i}{(p_i, Ap_i)}$$

with an **explicit** dot product, and the orthogonality it actually depends on,  $(r_i, p_{i-1}) = 0$ , holds *whatever*  $M^{-1}$  is. By induction, assuming  $(r_{i-1}, p_{i-2}) = 0$ :

$$(r_{i-1}, p_{i-1}) = (r_{i-1}, u_{i-1} + \beta_{i-1}p_{i-2}) = \gamma_{i-1}, \quad (r_i, p_{i-1}) = \gamma_{i-1} - \alpha_{i-1}(p_{i-1}, Ap_{i-1}) = 0,$$

with base case  $p_0 = u_0$ . So  $\alpha_i$  is always the exact line-search minimiser along  $p_i$ , and PCG remains a monotone descent method: it cannot diverge. What symmetry of  $M^{-1}$  buys is *full* conjugacy —  $r_i$  orthogonal to *all* previous directions, not just the last — i.e. the optimality of the short recurrence. Without it, convergence is slower, not absent.

Measured, plain reference PCG on a real CORE2 system ( $n_{\text{nz}} = 870\,146$ ), run to convergence with each preconditioner:

| preconditioner                  | plain-PCG iterations | $\sigma$ -recurrence drift |
|---------------------------------|----------------------|----------------------------|
| as built (non-symmetric)        | <b>126</b>           | $2.4 \times 10^{-1}$       |
| symmetrised $D^{-1/2}CD^{-1/2}$ | <b>117</b>           | $1.2 \times 10^{-13}$      |

**So the asymmetry costs the existing solver about 7 % in iterations** — small enough to have gone unnoticed, and nothing like a failure.

The Chronopoulos–Gear family is different. It *replaces* the explicit  $(p_i, Ap_i)$  with the recurrence  $\sigma_i = \delta_i - \beta_i^2 \sigma_{i-1}$ , whose derivation needs  $(r_i, M^{-\top} r_{i-1}) = 0$  — i.e. exactly the symmetry that is missing. The error then accumulates (21.8 % by iteration 60 on CORE2) with no line search to absorb it. In the model, `cg2` on the as-built preconditioner triggers the fallback guard on **18 of 20 solves**, the residual growing to  $2.9 \times 10^4$ ; with the symmetrised one it is clean.

#### 4. Suggested fix

Keep the diagonal (already  $1/d_i$ , which is what  $D^{-1/2}CD^{-1/2}$  gives) and scale each off-diagonal by  $\sqrt{d_i/d_j}$ :

$$\text{pr}[i, j] = -\frac{1}{2} \frac{a_{ij}}{\sqrt{d_i d_j} (d_i + d_j)} \quad (2)$$

```

DO n=2, nend
  node = cind(offset+n)
  ! symmetric form: -0.5*a_ij / ( sqrt(d_i*d_j) * (d_i+d_j) )
  pr_values(n+offset) = -0.5_WP*ssh_stiff%values(n+offset)/
                        ( sqrt(ssh_stiff%values(1+offset)*diag_values(node))
                        *(ssh_stiff%values(1+offset) + diag_values(node)) )
END DO

```

Three properties worth noting:

- **Symmetric** by inspection ( $a_{ij} = a_{ji}$ , and both  $\sqrt{d_i d_j}$  and  $d_i + d_j$  are symmetric).
- **Same spectrum.**  $D^{-1/2}CD^{-1/2} = D^{1/2}(D^{-1}C)D^{-1/2}$  is a similarity transform of the present operator, so the eigenvalues are unchanged — this is not a different preconditioner, it is the self-adjoint representative of the same one. (Checked numerically:  $\lambda_{\max}$  agrees to ten significant figures.)
- **Free.** Same sparsity, built once at setup, no per-iteration cost. Nothing downstream changes; `diag_values` is already halo-exchanged above.

#### 5. Caveats

- This is a **solver-class change to the production path**: the iterates differ within tolerance, so it deserves the usual convergence and climate check rather than a silent swap. We are holding our own variants to the same bar.
- $\sqrt{d_i d_j}$  is safe for realistic magnitudes here ( $d \sim 10^8$ , so the product is  $\sim 10^{16}$ , far inside double range). If that ever became a concern,  $\sqrt{d_i} \sqrt{d_j}$  is equivalent and cannot overflow.
- The similarity argument is about  $M^{-1}$  itself. The *preconditioned* operator  $M^{-1}A$  is not similar to the symmetrised one, and its extreme eigenvalue moves slightly (0.23 % on a variable-diagonal test problem) — same convergence class, not the identical operator.
- If the intended MITgcm construction is symmetric, the discrepancy entered at the leading  $1/d_i$  above, and it is worth checking against the original derivation — the form suggested here is the minimal spectrum-preserving repair, not necessarily the one originally intended.